ChitChatBusiness backend — step-by-step deploy runbook

A click-by-click guide to take **this** backend from a GitHub push to a live, verified deployment on AWS. Copy/paste the commands in order.

Everything here is specific to this repo:

- **Repo:** `https://github.com/Edhost-Pty-Ltd/ChitChatBusiness.git`
- **Backend path:** `server/` (the mobile app lives at the repo root)
- **Current branch:** `ChitchatBusiness_postgress`
- **Runtime:** Node 20, Fastify REST + Socket.IO, Postgres, Redis, S3, Firebase Admin
- **Image entry point:** `node dist/index.js`; migrations: `node dist/db/migrate.js up`

Placeholders you will replace:

Placeholder	Example	Meaning
<ACCOUNT_ID>	123456789012	Your AWS account id
<REGION>	us-east-1	AWS region (use one, everywhere)
<DOMAIN>	api.chitchat.example	Public API hostname

Set these once in your shell so the commands below just work:

```
export AWS_REGION=us-east-1
export ACCOUNT_ID=123456789012
export ECR_REPO=chitchat-backend
export IMAGE="$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$ECR_REPO"
```

Phase 0 — Prerequisites (once)

Install and verify these tools:

```
aws --version       # AWS CLI v2
docker --version    # Docker Desktop running
git --version
```

Authenticate the AWS CLI (`aws configure` with an IAM user/role that can manage ECR, ECS, RDS, ElastiCache, S3, IAM, Secrets Manager, ELB, Logs).

Have ready:

- The Firebase service-account JSON (currently `server/firebase-service-account.json`).
- A domain you control for `<DOMAIN>` (to put in front of the ALB).

Phase 1 — Push the backend to GitHub

The deploy code (Dockerfile, TLS support) is not committed yet. Commit **only** the backend changes so you don't sweep up unrelated app edits.

```
cd /Users/user/Desktop/ChitChatBusiness

# Stage the backend deploy additions + the TLS/migrate code changes.
git add \
  server/Dockerfile \
  server/.dockerignore \
  server/DEPLOYMENT_AWS.md \
  server/DEPLOY_RUNBOOK.md \
  server/.env.example \
  server/package.json \
  server/package-lock.json \
  server/src/config/env.ts \
  server/src/db/pool.ts \
  server/src/db/migrate.ts \
  server/src/redis/client.ts \
  server/src/jobs/queues.ts \
  server/src/jobs/scheduler.ts

git commit -m "backend: production Dockerfile + RDS/ElastiCache TLS support"
git push origin ChitchatBusiness_postgress
```

> Secrets are safe: `server/.env` and `server/firebase-service-account.json` are in `.gitignore` and excluded by `.dockerignore`, so they never reach GitHub or the image.

Checkpoint: the files appear in the GitHub branch and the push succeeds.

Phase 2 — Create the container registry (ECR)

```
aws ecr create-repository \
  --repository-name "$ECR_REPO" \
  --image-scanning-configuration scanOnPush=true \
  --region "$AWS_REGION"
```

Checkpoint: `aws ecr describe-repositories --repository-names "$ECR_REPO"` returns the repo.

Phase 3 — Build and push the image

The build happens from the `server/` directory (the Dockerfile lives there).

```
cd /Users/user/Desktop/ChitChatBusiness/server

# Log Docker in to ECR.
aws ecr get-login-password --region "$AWS_REGION" \
  | docker login --username AWS --password-stdin \
    "$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com"

# Build for the Fargate CPU architecture (x86_64).
docker build --platform linux/amd64 -t "$ECR_REPO:latest" .

# Tag + push.
docker tag "$ECR_REPO:latest" "$IMAGE:latest"
docker push "$IMAGE:latest"
```

Checkpoint: `aws ecr list-images --repository-name "$ECR_REPO"` lists the `latest` tag.

> ARM/Graviton note: if you want cheaper ARM tasks, build with `--platform linux/arm64` and set the task `runtimePlatform` to `ARM64` in Phase 7.

Phase 4 — Networking (VPC, subnets, security groups)

Use an existing VPC or create one with 2 public + 2 private subnets across two AZs and a NAT Gateway (so private tasks can reach ECR, Firebase, Expo, Gemini).

Create four security groups and wire them so only the next tier can reach each store:

```
VPC_ID=vpc-xxxxxxxx   # your VPC

# ALB: open 443 to the internet.
SG_ALB=$(aws ec2 create-security-group --group-name chitchat-alb \
  --description "ALB" --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG_ALB \
  --protocol tcp --port 443 --cidr 0.0.0.0/0

# App tasks: accept 4000 only from the ALB.
SG_APP=$(aws ec2 create-security-group --group-name chitchat-app \
  --description "Fargate tasks" --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG_APP \
  --protocol tcp --port 4000 --source-group $SG_ALB

# RDS: accept 5432 only from the app.
SG_RDS=$(aws ec2 create-security-group --group-name chitchat-rds \
  --description "Postgres" --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG_RDS \
  --protocol tcp --port 5432 --source-group $SG_APP

# Redis: accept 6379 only from the app.
SG_REDIS=$(aws ec2 create-security-group --group-name chitchat-redis \
  --description "Redis" --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG_REDIS \
  --protocol tcp --port 6379 --source-group $SG_APP
```

Record your two **private** subnet ids as `SUBNET_A` / `SUBNET_B` and two **public** subnet ids for the ALB.

Phase 5 — Data stores

5a. RDS PostgreSQL 16 (TLS enforced)

Create a DB subnet group (private subnets), then the instance. Enforce SSL with a parameter group where `rds.force_ssl = 1`. Put it in `SG_RDS`.

```
aws rds create-db-subnet-group \
  --db-subnet-group-name chitchat-db-subnets \
  --db-subnet-group-description "ChitChat private" \
  --subnet-ids $SUBNET_A $SUBNET_B

aws rds create-db-instance \
  --db-instance-identifier chitchat-pg \
  --engine postgres --engine-version 16 \
  --db-instance-class db.t4g.micro \
  --allocated-storage 20 \
  --master-username chitchat \
  --master-user-password 'CHANGE-ME-strong' \
  --db-name chitchat \
  --vpc-security-group-ids $SG_RDS \
  --db-subnet-group-name chitchat-db-subnets \
  --no-publicly-accessible --storage-encrypted
```

When it's available, note the endpoint. Your connection string becomes: `postgres://chitchat:CHANGE-ME-strong@<rds-endpoint>:5432/chitchat`

5b. ElastiCache for Redis 7 (TLS in transit)

Create the cache subnet group and a small replication group with **TransitEncryption enabled** (so the app connects with `rediss://`), in `SG_REDIS`.

```
aws elasticache create-cache-subnet-group \
  --cache-subnet-group-name chitchat-redis-subnets \
  --cache-subnet-group-description "ChitChat private" \
  --subnet-ids $SUBNET_A $SUBNET_B

aws elasticache create-replication-group \
  --replication-group-id chitchat-redis \
  --replication-group-description "ChitChat Redis" \
  --engine redis --cache-node-type cache.t4g.micro \
  --num-node-groups 1 --replicas-per-node-group 0 \
  --transit-encryption-enabled \
  --cache-subnet-group-name chitchat-redis-subnets \
  --security-group-ids $SG_REDIS
```

Note the primary endpoint !`rediss://<redis-endpoint>:6379`.

5c. S3 media bucket (private)

```
aws s3api create-bucket --bucket chitchat-media-prod --region "$AWS_REGION" \
  $( [ "$AWS_REGION" != us-east-1 ] && echo --create-bucket-configuration
LocationConstraint=$AWS_REGION )

aws s3api put-public-access-block --bucket chitchat-media-prod \
  --public-access-block-configuration \

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Create an IAM user (or role) scoped to this bucket and generate an access key — the app reads `S3_ACCESS_KEY_ID`/`S3_SECRET_ACCESS_KEY` in `src/media/presign.ts`. Keep the key for Phase 6.

Phase 6 — Secrets

Create one secret holding the sensitive values. Build the JSON safely from files so quoting of the Firebase key doesn't bite you:

```
FIREBASE_JSON=$(cat /Users/user/Desktop/ChitChatBusiness/server/firebase-service-
account.json)

aws secretsmanager create-secret --name chitchat/backend \
  --secret-string "$(jq -n \
    --arg db   'postgres://chitchat:CHANGE-ME-strong@<rds-endpoint>:5432/chitchat' \
    --arg redis 'rediss://<redis-endpoint>:6379' \
    --arg akid '<S3_ACCESS_KEY_ID>' \
    --arg asec '<S3_SECRET_ACCESS_KEY>' \
    --arg fb   "$FIREBASE_JSON" \
    --arg gem  '' \
    '{DATABASE_URL:$db, REDIS_URL:$redis, S3_ACCESS_KEY_ID:$akid, S3_SECRET_ACCESS_KEY:
$asec, FIREBASE_SERVICE_ACCOUNT_JSON:$fb, GEMINI_API_KEY:$gem}')"
```

Note the secret ARN (`aws secretsmanager describe-secret --secret-id chitchat/backend`).

> `GEMINI_API_KEY` is optional — leave it empty unless you use AI meeting minutes.

Phase 7 — ECS task definition

Create the IAM roles first:

- **Execution role** (`chitchat-exec`): attach `AmazonECSTaskExecutionRolePolicy` **plus** permission to read the secret (`secretsmanager:GetSecretValue` on the `chitchat/backend` ARN) and write logs.
- **Task role** (`chitchat-task`): permissions the app needs at runtime — `s3:PutObject/GetObject/DeleteObject` on `chitchat-media-prod/*`.

Create the CloudWatch log group:

```
aws logs create-log-group --log-group-name /ecs/chitchat-backend
```

Register the task definition. Save this as `taskdef.json` and fill in the ARNs / endpoints:

```
{
  "family": "chitchat-backend",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "512",
  "memory": "1024",
  "runtimePlatform": { "cpuArchitecture": "X86_64", "operatingSystemFamily": "LINUX" },
  "executionRoleArn": "arn:aws:iam::<ACCOUNT_ID>:role/chitchat-exec",
  "taskRoleArn": "arn:aws:iam::<ACCOUNT_ID>:role/chitchat-task",
  "containerDefinitions": [
    {
      "name": "chitchat-backend",
      "image": "<ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/chitchat-backend:latest",
      "essential": true,
      "portMappings": [{ "containerPort": 4000, "protocol": "tcp" }],
      "environment": [
        { "name": "NODE_ENV", "value": "production" },
        { "name": "PORT", "value": "4000" },
        { "name": "DATABASE_SSL", "value": "require" },
        { "name": "S3_REGION", "value": "<REGION>" },
        { "name": "S3_ENDPOINT", "value": "https://s3.<REGION>.amazonaws.com" },
        { "name": "S3_BUCKET", "value": "chitchat-media-prod" },
        { "name": "S3_FORCE_PATH_STYLE", "value": "false" },
        { "name": "CORS_ORIGINS", "value": "https://<DOMAIN>" },
        { "name": "FIREBASE_PROJECT_ID", "value": "chitchatbusiness-880ce" },
        { "name": "GEMINI_MODEL", "value": "gemini-3.8-flash" }
      ],
      "secrets": [
        { "name": "DATABASE_URL", "valueFrom":
"arn:aws:secretsmanager:<REGION>:<ACCOUNT_ID>:secret:chitchat/backend:DATABASE_URL::" },
        { "name": "REDIS_URL", "valueFrom":
"arn:aws:secretsmanager:<REGION>:<ACCOUNT_ID>:secret:chitchat/backend:REDIS_URL::" },
        { "name": "S3_ACCESS_KEY_ID", "valueFrom":
"arn:aws:secretsmanager:<REGION>:<ACCOUNT_ID>:secret:chitchat/backend:S3_ACCESS_KEY_ID::" },
        { "name": "S3_SECRET_ACCESS_KEY", "valueFrom":
"arn:aws:secretsmanager:<REGION>:<ACCOUNT_ID>:secret:chitchat/
backend:S3_SECRET_ACCESS_KEY::" },
        { "name": "FIREBASE_SERVICE_ACCOUNT_JSON", "valueFrom":
"arn:aws:secretsmanager:<REGION>:<ACCOUNT_ID>:secret:chitchat/
backend:FIREBASE_SERVICE_ACCOUNT_JSON::" }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/chitchat-backend",
          "awslogs-region": "<REGION>",
          "awslogs-stream-prefix": "app"
        }
      }
    }
  ]
}
```

```
aws ecs register-task-definition --cli-input-json file://taskdef.json
```

> The `:KEY::` suffix on each secret ARN pulls one JSON field out of the single secret — that's why the whole thing is one secret, not six.

Phase 8 — Load balancer (ALB) with sticky sessions

Socket.IO's polling!'WebSocket upgrade must land on the same task, so **stickiness is required**.

1. **ACM**: request a public certificate for `<DOMAIN>` in `<REGION>` and validate it via DNS (Route 53).

2. **ALB**: internet-facing, in the two **public** subnets, security group `SG_ALB`.
3. **Target group**: type `ip`, protocol HTTP, port **4000**, VPC `VPC_ID`.
   - Health check path `/health` (the app returns 200 only when Postgres and Redis are both reachable).
   - Enable **stickiness** (`lb_cookie`, ~1 hour).
4. **Listener**: HTTPS **443** using the ACM cert !' forward to the target groupSet the ALB **idle timeout to 300s** so idle WebSockets aren't cut.
5. (Optional) a port-80 listener that redirects to 443.

Point DNS: Route 53 `A`/`ALIAS` record for `<DOMAIN>` !' the ALB.

Phase 9 — Run database migrations (one-off task)

Run migrations **before** starting the service. Reuse the task definition with an overridden command:

```
aws ecs run-task \
  --cluster chitchat \
  --task-definition chitchat-backend \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[$SUBNET_A,
$SUBNET_B],securityGroups=[$SG_APP],assignPublicIp=DISABLED}" \
  --overrides '{"containerOverrides":[{"name":"chitchat-backend","command":["node","dist/db/
migrate.js","up"]}]}' \
  --region "$AWS_REGION"
```

(Create the cluster first if needed: `aws ecs create-cluster --cluster-name chitchat`.)

Checkpoint: in the CloudWatch log group `/ecs/chitchat-backend`, the one-off task logs `Migrations up complete` and exits 0. It's idempotent — safe to re-run.

Phase 10 — Create the ECS service

```
aws ecs create-service \
  --cluster chitchat \
  --service-name chitchat-backend \
  --task-definition chitchat-backend \
  --desired-count 2 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[$SUBNET_A,
$SUBNET_B],securityGroups=[$SG_APP],assignPublicIp=DISABLED}" \
  --load-balancers "targetGroupArn=<TG_ARN>,containerName=chitchat-
backend,containerPort=4000" \
  --health-check-grace-period-seconds 30 \
  --deployment-configuration "minimumHealthyPercent=100,maximumPercent=200" \
  --region "$AWS_REGION"
```

- `desired-count 2` gives HA across AZs and exercises the Redis adapter.
- Add target-tracking auto scaling on CPU ~60% (min 2) via Application Auto Scaling.
- Set the target group **deregistration delay to ~20s** so it lines up with the app's 15s graceful shutdown.

Checkpoint: the service reaches `runningCount = 2` and both targets show **healthy** in the target group.

Phase 11 — Verify the live deployment

```
# 1. Health endpoint through the ALB/TLS.
curl -s https://<DOMAIN>/health | jq
# expect: {"status":"ok","dependencies":{"postgres":true,"redis":true}}

# 2. TLS cert is served.
curl -sI https://<DOMAIN>/health | head -1   # HTTP/2 200
```

Then functionally:

- Sign in from the app with a real Firebase phone token ’ an authenticated REST call succeeds (not 401).
- Open two clients and send a message ’ it arrives in real time. With `desired-count 2`, this confirms the Redis adapter fans out across tasks.
- Upload media ’ a presigned S3 PUT succeeds (confirms `S3_FORCE_PATH_STYLE=false` and bucket perms).
- Watch `/ecs/chitchat-backend` logs for `ChitChat backend listening` and no repeated RDS/Redis connection errors.

Phase 12 — Point the mobile app at the backend

In the repo-root `.env` (consumed by the app, not the server):

```
EXPO_PUBLIC_API_URL=https://<DOMAIN>
EXPO_PUBLIC_SOCKET_URL=https://<DOMAIN>
```

Make sure every app origin is in the backend's `CORS_ORIGINS` (Phase 7). Rebuild the app so the new URLs are baked in.

Redeploying later (the short loop)

```
cd /Users/user/Desktop/ChitChatBusiness/server
# 1. commit + push your changes (Phase 1)
# 2. rebuild + push image (Phase 3)
docker build --platform linux/amd64 -t "$IMAGE:latest" . && docker push "$IMAGE:latest"
# 3. run migrations if the schema changed (Phase 9)
# 4. roll the service to the new image:
aws ecs update-service --cluster chitchat --service chitchat-backend \
  --force-new-deployment --region "$AWS_REGION"
```

ECS does a rolling replace at 100% min healthy; the app drains sockets/jobs on SIGTERM, so this is zero-downtime.

Optional: automate Phases 3 + 10 with GitHub Actions

If you want a push to `ChitchatBusiness_postgress` to build, push, and roll the service automatically, add a workflow that assumes an AWS role via OIDC, builds `server/`, pushes to ECR, runs the migration task, then `aws ecs update-service --force-new-deployment`. Ask and I'll generate `.github/workflows/deploy-backend.yml` wired to this repo.

Common pitfalls (specific to this backend)

- **503 from `/health`** ’ Postgres or Redis unreachable. Check `DATABASE_SSL` (RDS needs `require` or `verify`) and that `REDIS_URL` uses `rediss://` (TLS is on at ElastiCache). Confirm `SG_RDS`/`SG_REDIS` allow `SG_APP`.
- **Task exits immediately naming a variable** ’ that env var is missing. The server validates config on boot and exits 1 with the exact name. Add it to the task def `environment` or the secret.

- **Socket connects then drops / messages flaky across tasks** — enable ALB stickiness and raise idle timeout (Phase 8).
- `node-pg-migrate` **not found in the migration task** — already fixed: it's a runtime dependency now. Just rebuild the image.
- **401 on every request** — the Firebase service-account JSON in the secret is malformed. Recreate the secret from the file as shown in Phase 6.